

Java Persistence API (JPA) y Spring Boot.

Configuración Inicial del Proyecto

- **Dependencias [02:31]:** Se utiliza Spring Initializr para generar un proyecto con Java 17 y Maven. Las dependencias seleccionadas son:
 - **Spring Web**
 - **MySQL Driver** (para la conexión a base de datos)
 - **Spring Data JPA** (para la capa de persistencia)
 - **Lombok** (para reducir código repetitivo como getters/setters).
- **Configuración de Propiedades [04:16]:** En `application.properties` se define la cadena de conexión JDBC, usuario, contraseña, dialecto de Hibernate y propiedades para mostrar el SQL generado (`show-sql`) y para la actualización automática del esquema (`ddl-auto=update`).

Implementación de Entidades y Relaciones

El video utiliza un dominio de **Fútbol** para ejemplificar las relaciones.

1. Relación Uno a Uno (One-to-One) [11:15]

- **Escenario:** Un **Club** tiene un **Coach** (Entrenador), y un entrenador pertenece a un solo club.
- **Implementación:**
 - Se crea la entidad **Coach** con atributos (`id`, `name`, `lastName`, `age`, `nationality`). Se usa `@Column(name = "last_name")` para mapear *camelCase* de Java a *snake_case* de SQL.
 - En la entidad **Club**, se agrega el campo `private Coach coach`.
 - Se anota con `@OneToOne`.
 - **Configuración de la Relación:**
 - `targetEntity = Coach.class`: Define la clase destino.
 - `cascade = CascadeType.PERSIST`: Define que al guardar el club, se guarde también el entrenador (comportamiento en cascada) [18:05].
 - **Clave Foránea:** Se discute la ubicación de la *Foreign Key*. Se decide colocarla en **Club** ya que es la entidad principal de consulta. Se personaliza el nombre de la columna con `@JoinColumn(name = "id_coach")` [21:33].

2. Relación Uno a Muchos (One-to-Many) [22:30]

- **Escenario:** Un **Club** tiene muchos **Players** (Jugadores), pero un jugador pertenece a un solo club.
- **Implementación Bidireccional:**
 - **Lado "Uno" (Club):** Se añade una lista `private List<Player> players`. Se anota con `@OneToMany`.
 - **Lado "Muchos" (Player):** Se añade `private Club club`. Se anota con `@ManyToOne`.
- **Corrección de Tabla Intermedia [33:05]:** Inicialmente, JPA genera erróneamente una tabla intermedia. Para corregirlo y seguir el estándar SQL (la *Foreign Key* va en la tabla del "Muchos"), se usa el atributo `mappedBy = "club"` en la anotación `@OneToMany` de **Club**. Esto indica que la relación es gestionada por el campo `club` en la entidad **Player**.
- **Fetch Type [29:01]:** Se explica la diferencia entre **EAGER** (carga ansiosa) y **LAZY** (carga perezosa). Se configura `FetchType.LAZY` para la lista de jugadores por razones de rendimiento.

3. Relación Muchos a Uno (Many-to-One) [36:00]

- **Escenario:** Muchos **Club** pertenecen a una **FootballAssociation** (Asociación de Fútbol).
- **Implementación:**
 - Es la inversa de la anterior, vista desde la perspectiva de la asociación.
 - **FootballAssociation** tiene una lista de clubes (`@OneToMany` con `mappedBy`).
 - **Club** tiene una referencia a **FootballAssociation** anotada con `@ManyToOne`.
 - Se verifica en base de datos que la tabla **Club** ahora contiene la columna `football_association_id` como clave foránea [43:56].

4. Relación Muchos a Muchos (Many-to-Many) [44:46]

- **Escenario:** Un **Club** participa en muchas **FootballCompetition** (Competiciones), y una competición tiene muchos clubes.
- **Implementación:**
 - Se crea la entidad **FootballCompetition**.
 - En **Club**, se añade `List<FootballCompetition>` con `@ManyToMany`.
 - En **FootballCompetition**, se podría añadir la lista inversa.
- **Tabla Intermedia [58:28]:** JPA genera automáticamente una tabla intermedia para gestionar la relación.
- **Personalización con @JoinTable [59:00]:** Se personaliza la tabla de unión explícitamente:
 - `name = "club_competition"`: Nombre de la tabla intermedia.
 - `joinColumns`: Configura la FK de la entidad dueña (`@JoinColumn(name = "club")`).

- `inverseJoinColumn`: Configura la FK de la entidad inversa (`@JoinColumn(name = "competition")`).

Detalles Técnicos Adicionales

- **Anotación `@Column` [48:33]**: Análisis profundo de sus atributos:
 - `length`: Longitud máxima (ej. `VARCHAR(10)`).
 - `nullable`: Si permite NULL (default true).
 - `unique`: Restricción de unicidad.
 - `updatable` / `insertable`: Control sobre operaciones DML.
 - `columnDefinition`: Definición manual del tipo SQL (ej. "DATE", "VARCHAR(300)") [51:39].
- **Ingeniería Inversa [01:02:47]**: Uso de la función *Reverse Engineer* en MySQL Workbench para generar visualmente el diagrama Entidad-Relación (ER) a partir de las tablas generadas por JPA, validando la cardinalidad y las claves foráneas creadas.